

V1 — Tool Management Workspace & Agent Configuration Extensions

Desktop-side implementation notes for the Gabriel V1 resource-model milestone.

This is Part 2 of the V1 work: it consumes the gabriel-core APIs added on

`feature/v1-tool-knowledge-agent-config` (core PR #8) — the `/api/v1/tools`

CRUD endpoints, `source_type` on knowledge sources, and the agent

`disabled_tools` / `document_collections` configuration fields.

Scope

1. A new **Tools workspace** for browsing, searching, filtering, enabling/disabling, inspecting, and configuring tools.
2. **Agent editor extensions** — assign/remove tools, knowledge sources, and document collections on an agent.
3. **Navigation** — Tools added to the workspace nav (sidebar + command palette, which both render from the single nav config per ADR-035).
4. **API integration** — typed services against gabriel-core, following the existing DTO → domain mapping pattern (`gateway-client` → service → hooks → view).

Changes by layer

Types

File	Change
src/types/tool.ts	New. <code>Tool</code> domain type plus <code>ToolCategory</code> , <code>ExecutionRuntime</code> , <code>SafetyLevel</code> unions mirroring gabriel-core's enums. <code>TOOL_CATEGORIES</code> / <code>EXECUTION_RUNTIMES</code> constants drive filter chips and selects.
src/types/knowledge.ts	Added <code>KnowledgeSourceType</code> (<code>vector_collection</code> <code>document_collection</code> <code>external</code>) and <code>sourceType</code> on <code>KnowledgeSource</code> .
src/types/agent.ts	<code>Agent</code> gains <code>documentCollections</code> and <code>disabledTools</code> .
src/types/api.ts	New <code>ToolDto</code> / <code>ToolCreateDto</code> / <code>ToolUpdateDto</code> wire shapes; <code>AgentDto</code> / <code>AgentCreateDto</code> gain <code>disabled_tools</code> + <code>document_collections</code> ; <code>KnowledgeSourceDto</code> gains <code>source_type</code> . Optional where older core deployments may omit the field.
src/types/workspace.ts	<code>'tools'</code> added to <code>WorkspaceId</code> .

Services (API integration)

File	Change
<code>src/services/tools.ts</code>	New. <code>listTools</code> (server-side <code>category / enabled / execution_runtime</code> filters), <code>getTool</code> , <code>createTool</code> , <code>updateTool</code> (PATCH), <code>setToolEnabled</code> (PATCH { <code>enabled</code> }), <code>deleteTool</code> — all against <code>/api/v1/tools</code> , via the shared <code>gatewayRequest</code> client (bearer auth, 401 refresh, <code>GatewayError</code>).
<code>src/services/agents.ts</code>	<code>mapAgent</code> maps <code>disabled_tools / document_collections</code> ; <code>UpdateAgentInput</code> gains <code>documentCollections</code> , <code>disabledTools</code> , <code>allowedTools</code> which serialize to the core agent PATCH wire names.
<code>src/services/knowledge.ts</code>	<code>mapSource</code> maps <code>source_type</code> (defensive default <code>vector_collection</code>); <code>listKnowledgeSources</code> accepts a <code>sourceType</code> filter (<code>?source_type=</code>); <code>createKnowledgeSource</code> accepts <code>sourceType</code> .
<code>src/services/index.ts</code>	Exports the new <code>tools</code> namespace.

Tools workspace

- `src/app/(app)/tools/page.tsx` — route under the authenticated `(app)` layout, identical wiring to `/agents`.
- `src/features/tools/hooks.ts` — TanStack Query hooks (`useTools`, `useCreateTool`, `useUpdateTool`, `useSetToolEnabled`, `useDeleteTool`) keyed on `['tools', filters]` with invalidation on every mutation, matching the agents feature.
- `src/features/tools/tools-view.tsx` — card grid styled after Agent Management:
- Per-category lucide icon tile, name, description, category / runtime / safety badges, runtime-binding footer with resource version.
- Search box (name + description), enabled-state chips (all/enabled/disabled), and category filter chips. Category filtering is pushed to the server (`GET /tools?category=`); search/status refine client-side for instant feedback.
- Inline **enable/disable switch** on every card (org-level `enabled`, deny-wins across all agents — matches core's `ChatRuntimeService`).
- **Inspect** dialog: GRN, badges, runtime binding, required capabilities, version, created/updated dates, input/output schemas, and configuration rendered as JSON blocks.
- **Register** and **Configure** dialogs share one form component: name, description, category, execution runtime, safety level, runtime binding,

and a validated configuration JSON editor (invalid JSON blocks submit with an inline message).

- Delete confirmation dialog mirroring the agent delete flow.

Agent editor extensions (`src/features/agents/agent-detail.tsx`)

The draft/save model is unchanged (single dirty-checked **Save**); three assignment surfaces were added without new chrome:

- **Tools card** (main column): lists all org tools. Clicking a row assigns/unassigns the tool (`allowed_tools`); assigned rows show an Allowed/Blocked switch that toggles the tool's presence in `disabled_tools` (per-agent deny, deny-wins in core). Unassigning a tool also clears its per-agent deny. Org-wide disabled tools are flagged with a badge. Helper copy states the core semantics: no assigned tools = agent can use every enabled tool.
- **Knowledge sources card** (side column): now scoped to `sourceType !== 'document_collection'` (vector collections), same toggle-to-link interaction as before → `knowledge_sources` .
- **Document collections card** (side column, new): typed `document_collection` sources, same interaction → `document_collections` .

All four lists are sent on save through the existing agent PATCH.

Documents page

The New knowledge source dialog gained a **Type** select (vector collection / document collection) so document collections can be created from the UI and then assigned to agents.

Navigation

`src/config/navigation.ts` : Tools workspace registered between Agents and Documents (`/tools` , `Wrench` icon, shortcut `g t` , priority `p0` , enabled). Sidebar, breadcrumbs, and command palette all render from this single config (ADR-035), so no other nav surface needed changes.

Shared helpers

`src/lib/format.ts` gained `formatDate` (absolute date for metadata panels).

ADR compliance

- **ADR-005 / ADR-009 / ADR-016 (Universal Resource / GRN)**: tools are displayed and addressed by GRN; the inspect dialog surfaces GRN, version, and audit timestamps. No client-side identity invented.
- **ADR-007 §7 (per-agent LLM config)**: untouched; tool/knowledge assignment extends the same per-agent specification.
- **ADR-008 (knowledge abstraction)**: the UI treats `source_type` as an opaque server-defined discriminator with a defensive default; document collections are just typed knowledge sources, exactly as core models them.
- **ADR-035 (workspace navigation)**: the new workspace is declared once in the nav config; every nav surface updates from it.

- Services remain the only HTTP layer (`gateway-client` pattern); components never fetch directly.

Not in scope (matches core V1)

- `execution_runtime` is a declaration only — the UI labels it as such and no routing behavior is implied.
- `external` knowledge sources are typed but not creatable from the UI (no connector support in core yet).
- Tool invocation/testing UI — the chat runtime already streams `tool_call` / `tool_result` frames; a tool playground is future work.

Validation

- `pnpm typecheck` — clean.
- `pnpm lint` — no ESLint warnings or errors.
- `pnpm build` — production build succeeds; `/tools` route emitted alongside existing routes.